Deployment of post-quantum cryptography: the threat of side-channel attacks

Simon Abelard
Associate professor, LRE, EPITA (Paris)
Associate researcher, LIP6, Sorbonne (Paris)

Fall 2026

Short bio

Vita:

- 5 years in academia (PhD, postdoc)
- 5 years at Thales (crypto, cyber)
- Back in academia

Academic activities:

- Computer algebra, algorithms, complexity
- Algebraic cryptanalysis (multivariate)
- Protocols (both hybrid & quantum)

Short bio

Vita:

- 5 years in academia (PhD, postdoc)
- 5 years at Thales (crypto, cyber)
- Back in academia

Academic activities:

- Computer algebra, algorithms, complexity
- Algebraic cryptanalysis (multivariate)
- Protocols (both hybrid & quantum)

Deployment of PQ crypto:

- Prototyping algorithms (software and hardware)
- Hardening implementations vs side-channels
- Designing secure protocols for space systems (satellites and ground to space link)
- Defining security architecture for complex systems
- Proposing new primitives (Broadcast Encryption)

Course outline

- **Generalities:**
  - Quick recall from L. Perret's course
  - Code-based cryptography
- **Side-channel attacks:**
  - Taxonomy and examples
  - Basic attacks on multiplication and formatting
- **More sophisticated attacks:**
  - On lattice-based schemes
  - On code-based schemes
- **Beyond side-channel attacks:**
  - Countermeasures: masking and shuffling
  - Fault injection attacks
  - Cryptanalysis

Grading process

- **Final grade:**
  - 40% = Cryptographic design (group activity)
  - 60 % = Final exam
- **Final exam**
  - In December (TBC), 2 hours
  - No authorized material
  - Evaluate ability to safely use and deploy cryptography
  - Technical & complicated elements will be recalled (e.g. no need to remember every formula by heart)

Why study cryptography ?

The weakest link is almost always the user and almost never the cryptographic algorithms, BUT

- Not studying cryptography is taking the risk to make it the weakest link!
- A cryptographic failure is less likely but can have a global (and catastrophic) impact

Why study cryptography ?

The weakest link is almost always the user and almost never the cryptographic algorithms, BUT

- Not studying cryptography is taking the risk to make it the weakest link!
- A cryptographic failure is less likely but can have a global (and catastrophic) impact

Examples: Jailbreak of PS3 (bad implementation of cryptography), LOGJAM attack targeting **8% of the Internet** (poorly dimensioned cryptography), breaking ENIGMA during WW2 (misuse and bad dimensioning)

A brief introduction to post-quantum cryptography

Security of public-key cryptography (PKC)

Hard problems: PKC relies on one-way functions based on allegedly hard mathematical problems such as the discrete logarithm and factorization.

Computational security: our definition of hard is "that takes an unreasonable amount of time and resource".

Security of public-key cryptography (PKC)

Hard problems: PKC relies on one-way functions based on allegedly hard mathematical problems such as the discrete logarithm and factorization.

Computational security: our definition of hard is "that takes an unreasonable amount of time and resource".

One-wayness in practice

In 1s, a laptop from 2015 can either perform

- the product of two 30-million-digit integers
- the factorization of a 42-digit integer

As of today, nobody ever factored a number with > 260 decimal digits. Previous record (RSA-250) took 2700 core-years.
Currents standards use numbers of > 700 digits.

Quantum computing

The notion of hardness depends on the tools at hand (hardware and algorithms). Quantum computing $\rightsquigarrow$ new tools!

Some quantum algorithms offer limited speed-up (Grover, 1996), some offer exponential speed-up (Shor, 1994).

Cryptographic consequences:

- Symmetric crypto: theoretically safe after doubling key size.
  NIST considers Grover's algorithm non practical.
  NIST: AES-128 is OK vs NÚKIB: switch to AES-256
- Public-key crypto: both RSA and (EC)DH will be broken by a sufficiently large quantum computer.

Which post-quantum algorithm should I use?

General purpose: structured lattices

- Kyber aka ML-KEM (NIST FIPS 203) for encryption
- Dilithium aka ML-DSA (NIST FIPS 204) for signature

Conservative / back up options:

- HQC (code-based) future NIST standard, competitive
- FrodoKEM (unstructured lattices), less competitive but simple to implement, supported by BSI, ANSSI
- SPHINCS+ aka SLH-DSA (hash-based), NIST FIPS 205, huge but very conservative in terms of security

Which post-quantum algorithm should I use?

General purpose: structured lattices

- Kyber aka ML-KEM (NIST FIPS 203) for encryption
- Dilithium aka ML-DSA (NIST FIPS 204) for signature

Conservative / back up options:

- HQC (code-based) future NIST standard, competitive
- FrodoKEM (unstructured lattices), less competitive but simple to implement, supported by BSI, ANSSI
- SPHINCS+ aka SLH-DSA (hash-based), NIST FIPS 205, huge but very conservative in terms of security

Specific use cases:

- Falcon (future FIPS 206) when bandwidth is limited, but implementation / side-channel issues
- XMSS (hash-based) for software/firmware signing

Code-based and lattice-based cryptography

Quantum-hard problems

Security of asymmetric cryptography

Remember: we need **hard mathematical problems** that also resist quantum computers.

Examples:

- Finding short/close vectors in lattices
- Decoding random error-correcting codes
- Inverting hash functions
- Solving quadratic equations in many variables
- And many more...

Foundation of code-based cryptography

Quick recall:

- Shannon's theorem $\rightsquigarrow$ we can generally decode.
- Shannon's theorem does not say that it is easy!
- Actually it can be quite hard...

Bounded-distance decoding (BDD)

Given as input $C \subset \mathbb{F}_q^n$, $y \in \mathbb{F}_q^n$, $0 \leq t \leq n$,
Determine whether there is $c \in C$ s.t. $d_H(y, c) \leq t$. (y close to c)

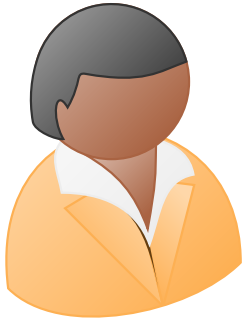
Berlekamp, McEliece, van Tilbørg, 1978

It is (in general) hard to compute c from y, i.e. **decoding is hard**.

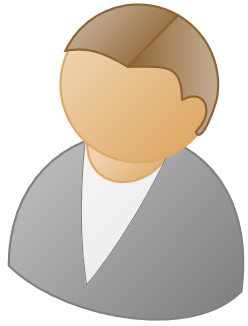
McEliece's cryptosystem

Public key (G, t)

Message m



Alice



Bob

Private key

Decode

G generates the code, t the error weight

McEliece's cryptosystem

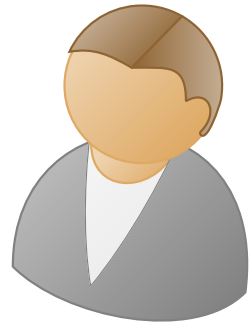
Public key (G, t)

Message m

e of weight t



Alice



Bob

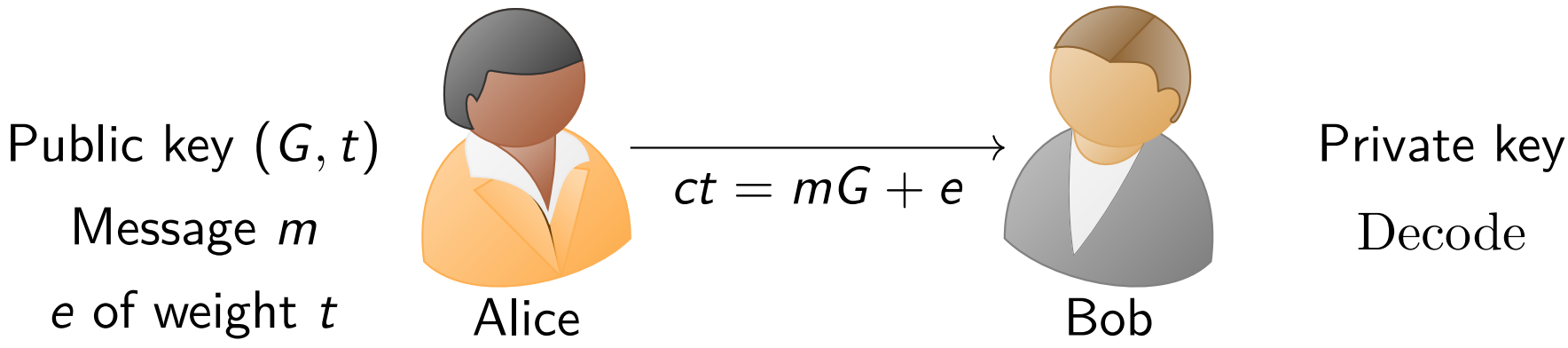
Private key

Decode

G generates the code, t the error weight

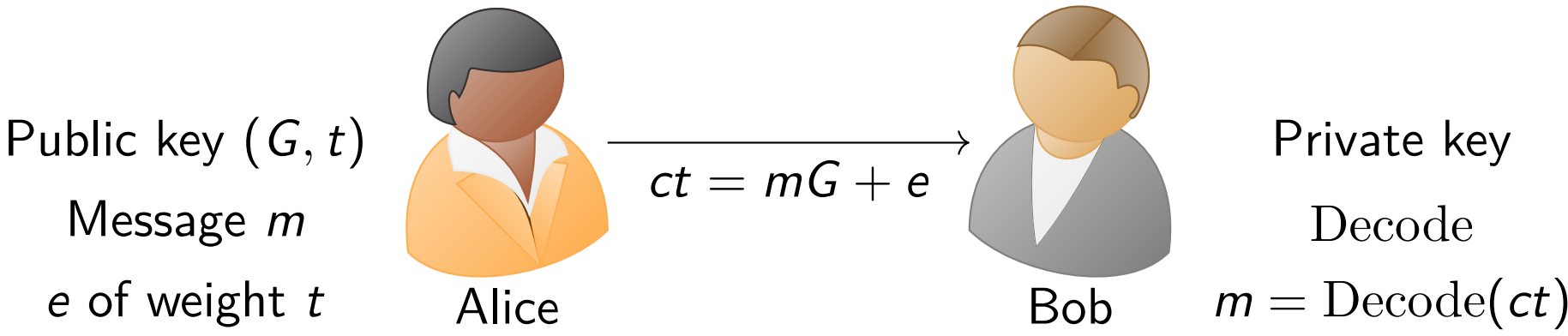
Encryption $=$ add noise to mG (codeword) $\rightsquigarrow$ decryption $=$ decoding

McEliece's cryptosystem



G generates the code, t the error weight
Encryption = add noise to mG (codeword) $\rightsquigarrow$ decryption = decoding

McEliece's cryptosystem



G generates the code, t the error weight
Encryption = add noise to mG (codeword) $\rightsquigarrow$ decryption = decoding

McEliece's cryptosystem



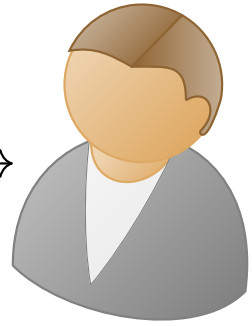
Public key (G, t)

Message m

e of weight t

Alice

$ct = mG + e$



Bob

Private key

Decode

$m = \mathrm{Decode}(ct)$



Eve

Knows (G, t)

Intercept ct

Can decode?

G generates the code, t the error weight

Encryption = add noise to mG (codeword) $\rightsquigarrow$ decryption = decoding

Hardness of decoding $\Rightarrow$ only Bob can recover m

More than a cryptosystem, a framework

Parameter choice

Each pair (code, distance) brings a new cryptosystem
This choice is essential to ensure both performance and security!

- Broken codes: Reed-Solomon, BCH, low-density quasi-cyclic
- NIST finalist: Goppa codes (Classic McEliece)
  Broken in August (superpolynomial classical attack)
- NIST standard: HQC = Hamming Quasi Cyclic

Codes in the NIST competition

Classic McEliece: small cipher (128 B), huge pub. key ($>$ 200 kB).
BIKE & HQC: slightly worse than lattices but good options.
Wave: short signature but huge public key ($>$ 3 MB).

Quasi Cyclic Moderate Density (QC-MDPC) codes

Main drawback of codes: public key is a huge matrix.

Circulant matrice: matrice described by a vector $(a_0, \ldots, a_{n-1})$.

$$\begin{pmatrix} a_0 & a_1 & \ldots & a_{n-1} \\ a_1 & a_2 & \ldots & a_0 \\ \vdots & \vdots & & \vdots \\ a_{n-1} & a_0 & \ldots & a_{n-2} \end{pmatrix}$$

Quasi-cyclic codes: codes described by blocks of circulant matrices $\rightsquigarrow$ public key + ciphertext = a few vectors.

Quasi Cyclic Moderate Density (QC-MDPC) codes

Main drawback of codes: public key is a huge matrix.

Circulant matrice: matrice described by a vector $(a_0, \ldots, a_{n-1})$.

$$\begin{pmatrix} a_0 & a_1 & \ldots & a_{n-1} \\ a_1 & a_2 & \ldots & a_0 \\ \vdots & \vdots & & \vdots \\ a_{n-1} & a_0 & \ldots & a_{n-2} \end{pmatrix}$$

Quasi-cyclic codes: codes described by blocks of circulant matrices $\rightsquigarrow$ public key + ciphertext = a few vectors.

Moderate density: each row of the parity-check matrix has n entries of which only $w = O(\sqrt{n})$ are non-zero.

Does this decrease security? So far, not really.
(there are attacks providing a gain sub-linear in n).

From circulant matrices to polynomials

Consider $P = a_0 + a_1 X + \cdots a_{n-1} X^{n-1}$ modulo $X^n - 1$, then

$$X \times P = a_{n-1} + a_0 X + a_1 X^2 + \cdots + a_{n-2} X^{n-1}.$$

Circulant matrix = representing $X^i \times P$ in basis $1, X, \ldots X^{n-1}$.

An isomorphism: the ring of $n \times n$ circulant matrices over $\mathbb{F}$ is isomorphic to $\mathbb{F}[X]/(X^n - 1)$.

Transposition: we can define $P^T = a_0 + a_{n-1} X + \cdots a_1 X^{n-1}$ so that the above isomorphism commutes with transposition.

From circulant matrices to polynomials

Consider $P = a_0 + a_1 X + \cdots a_{n-1} X^{n-1}$ modulo $X^n - 1$, then

$$X \times P = a_{n-1} + a_0 X + a_1 X^2 + \cdots + a_{n-2} X^{n-1}.$$

Circulant matrix = representing $X^i \times P$ in basis $1, X, \ldots X^{n-1}$.

An isomorphism: the ring of $n \times n$ circulant matrices over $\mathbb{F}$ is isomorphic to $\mathbb{F}[X]/(X^n - 1)$.

Transposition: we can define $P^T = a_0 + a_{n-1} X + \cdots a_1 X^{n-1}$ so that the above isomorphism commutes with transposition.

Conclusion: operations on vectors and circulant matrices become operations on polynomials modulo $(X^n - 1)$.

Parameter choice: for performance and security, work in $\mathbb{F}_2[X]/(X^n - 1)$ with n prime and 2 primitive modulo n.
(i.e. $2^k = 1 \bmod n$ iff $(n-1)$ divides k)

The KEM BIKE (BIt-flipping Key Encapsulation)

Private key: $h_0, h_1 \in \mathbb{F}_2[X]/(X^n - 1)$ of Hamming weight w.
Public key: $h = h_1 h_0^{-1}$ in $\mathbb{F}_2[X]/(X^n - 1)$.

Encryption: message $m = (e_0, e_1)$ with total Hamming weight t.
Ciphertext is $c = e_0 + e_1 h$.

Decryption: first, compute $h_0 c = h_0 e_0 + e_1 h_1$.
Then, $\mathrm{Decode}(h_0 c, h_0, h_1, t)$ yields $m = (e_0, e_1)$.

The KEM BIKE (BIt-flipping Key Encapsulation)

Private key: $h_0, h_1 \in \mathbb{F}_2[X]/(X^n - 1)$ of Hamming weight w.
Public key: $h = h_1 h_0^{-1}$ in $\mathbb{F}_2[X]/(X^n - 1)$.

Encryption: message $m = (e_0, e_1)$ with total Hamming weight t.
Ciphertext is $c = e_0 + e_1 h$.

Decryption: first, compute $h_0 c = h_0 e_0 + e_1 h_1$.
Then, $\mathrm{Decode}(h_0 c, h_0, h_1, t)$ yields $m = (e_0, e_1)$.

Why? $(h_0, h_1) \rightsquigarrow$ quasi-circulant matrix $H = (H_0 \quad H_1)$.
Then $hc_0 = (e_0, e_1)H^T$ is called a **syndrome** of e.
Decoder: (syndrome, matrix H) $\mapsto$ error (e_0, e_1).

Remarks:

- Decoding is hard if h_0, h_1 (i.e. H) are unknown
- Here the message is the error (Niederreiter vs McEliece)

The KEM HQC (Hamming Quasi-Cyclic)

Similarities with BIKE:
Keys: $sk = (x, y)$ of weight w, $pk = x + hy$ with h random.

Differences: McEliece framework + another layer (code $\mathcal{C}$).

Encrypt: $ct = (u, v)$, $u = r_1 + hr_2$, $v = \mathrm{Encode}_{\mathcal{C}}(m) + pkr_2 + e$.

Decrypt: Compute $v - uy = \mathrm{Encode}_{\mathcal{C}}(m) + xr_2 + e - r_1 y$.
If $xr_2 + e - r_1 y$ has small weight, $\mathrm{Decode}_{\mathcal{C}}(v - uy) = m$.

Concept of HQC:

- Someone knowing (x, y) will only have to decode $\mathcal{C}$.
- Anyone else must decode the QC-MDPC before.
- The code $\mathcal{C}$ need not be hard to decode!

Decoding methods

For BIKE: BGF (black gray flip decoder)
From arbitrary $e = (e_0, e_1)$, look at equation $hc_0 = eH^T$.
Identify bits of e causing $hc_0 \neq eH^T$, flip them and repeat.
Conceptually simple decoder but hard to analyze (security issue).

For HQC: decoding is done on $\mathcal{C}$ which is made of "easy" codes (Reed-Solomon and Reed-Muller), hence no need to decode the quasi-cyclic part.

Decoding methods

For BIKE: BGF (black gray flip decoder)
From arbitrary $e = (e_0, e_1)$, look at equation $hc_0 = eH^T$.
Identify bits of e causing $hc_0 \neq eH^T$, flip them and repeat.
Conceptually simple decoder but hard to analyze (security issue).

For HQC: decoding is done on $\mathcal{C}$ which is made of "easy" codes (Reed-Solomon and Reed-Muller), hence no need to decode the quasi-cyclic part.

Importance in design: while BIKE is conceptually simpler, it is much easier to model the decoding of HQC and thus predict the decoding failures and their impact on security.

Decoding failure? We will come back to it later.

Differences in performance: HQC use more bandwidth but it is faster than BIKE.

Lattice-based cryptography

Lattices

Given a ring R, a lattice L over R^n is the set of integer linear combinations of m linearly independent vectors $b_1, \ldots, b_m$.

$$L = L(b_1, \ldots, b_m) = \left\{ \sum_{i=1}^{m} \alpha_i b_i \,\middle|\, (\alpha_1, \ldots, \alpha_m) \in \mathbb{Z}^m \right\}$$

Base ring: most common are $R = \mathbb{Z}_q$ or $R = \mathbb{Z}_q[X]/(X^n + 1)$.

Definitions: n is the *dimension* and m the *rank* of L.

Basis: $B = (b_1, \ldots, b_m)$ is called a *basis* of L.

We will see that not all bases are equally convenient: a good basis usually have vectors b_i of same norm and orthogonal.

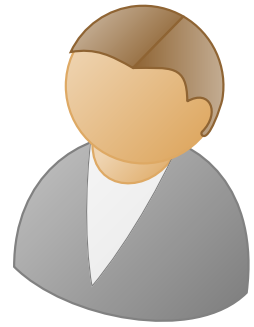
Kyber, a lattice based cryptosystem

Public key (A, t)

Message m



Alice



Bob

Private key

e, s s.t.
$t = As + e$

A is a matrix, e a random error

Kyber, a lattice based cryptosystem

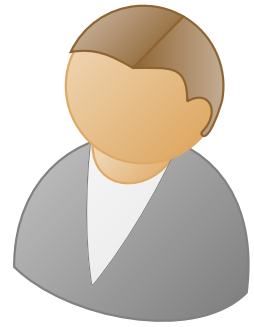
Public key (A, t)

Message m

Small e_1, e_2



Alice



Bob

Private key

e, s s.t.
$t = As + e$

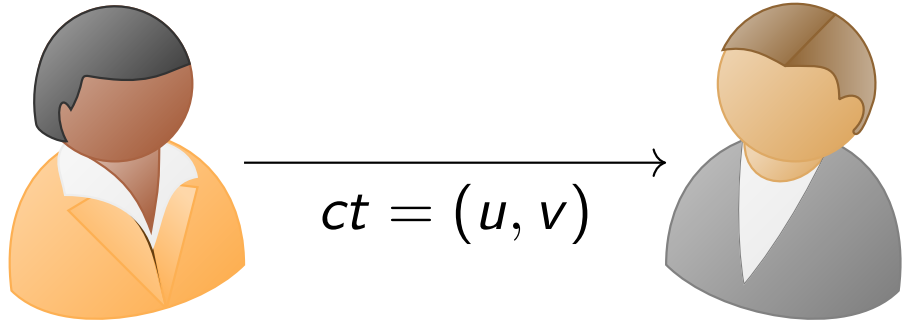
A is a matrix, e a random error
Encryption $=$ generate random r and small e_1, e_2 (e.g. coefs $\in [-\eta, \eta]$),
Compute $u = A^T r + e_1$ and $v = t^T r + e_2 + \lceil \frac{q}{2} \rfloor m$
Decryption $=$ kills r and error via $m = \mathrm{Round}(v - s^T u)$

Kyber, a lattice based cryptosystem

Public key (A, t)

Message m

Small e_1, e_2



Private key

e, s s.t.
$t = As + e$

A is a matrix, e a random error

Encryption $=$ generate random r and small e_1, e_2 (e.g. coefs $\in [-\eta, \eta]$),

Compute $u = A^T r + e_1$ and $v = t^T r + e_2 + \lceil \frac{q}{2} \rfloor m$

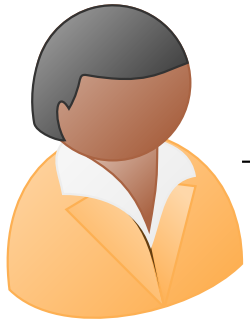
Decryption $=$ kills r and error via $m = \mathrm{Round}(v - s^T u)$

Kyber, a lattice based cryptosystem

Public key (A, t)

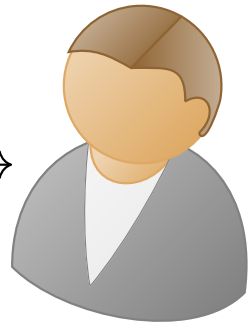
Message m

Small e_1, e_2



Alice

$$ct = (u, v)$$



Bob

Private key

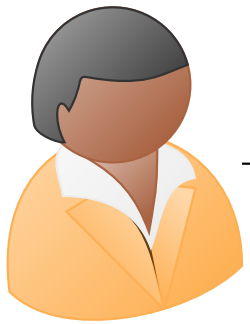
e, s s.t.
$t = As + e$

$$m' = v - s^T u$$

$$m = \mathrm{Round}(m')$$

A is a matrix, e a random error
Encryption $=$ generate random r and small e_1, e_2 (e.g. coefs $\in [-\eta, \eta]$),
Compute $u = A^T r + e_1$ and $v = t^T r + e_2 + \lceil \frac{q}{2} \rfloor m$
Decryption $=$ kills r and error via $m = \mathrm{Round}(v - s^T u)$

Kyber, a lattice based cryptosystem



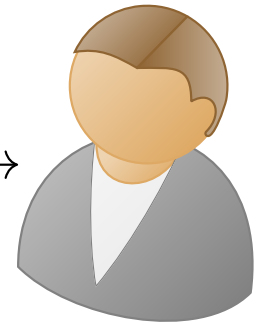
Alice

Public key (A, t)

Message m

Small e_1, e_2

$ct = (u, v)$



Bob

Private key

e, s s.t.
$t = As + e$

$m' = v - s^T u$

$m = \mathrm{Round}(m')$



Eve

Knows (A, t)

Intercepts ct

Can get m?

A is a matrix, e a random error

Encryption $=$ generate random r and small e_1, e_2 (e.g. coefs $\in [-\eta, \eta]$),

Compute $u = A^T r + e_1$ and $v = t^T r + e_2 + \lceil \frac{q}{2} \rfloor m$

Decryption $=$ kills r and error via $m = \mathrm{Round}(v - s^T u)$

Hardness of module LWE $\Rightarrow$ only Bob can recover m

Why it should be familiar

Kyber's idea:

- Create random r
- $u = A^T r + e_1$ bears info on r
- $v = t^T r + e_2 + \lceil \frac{q}{2} \rfloor m$ has both r and m
- Knowledge of s allows to eliminate r and get m

Why it should be familiar

Kyber's idea:

- Create random r
- $u = A^T r + e_1$ bears info on r
- $v = t^T r + e_2 + \lceil \frac{q}{2} \rfloor m$ has both r and m
- Knowledge of s allows to eliminate r and get m

A classical design: This was done already by El Gamal!
$ct = (c_1, c_2) = (g^r, mh^r)$, with r random and h is the public key.

This is also done in HQC with h disappearing from $v - uy$.

What about e_2? It is an optimization that allows to encrypt longer messages without increasing the size of A or t.

Food for thought: lattices and codes

Lattice-based schemes	**Code-based schemes**
Encaps: add small error	Encaps: $ct = mG + e$
Decaps: rounding to kill e_i	Decaps: $m =$ codeword closest to ct

Question: are lattices and code the same?

Distance: Euclidean (lattices) vs Hamming (codes).

Food for thought: lattices and codes

Lattice-based schemes	**Code-based schemes**
Encaps: add small error	Encaps: $ct = mG + e$
Decaps: rounding to kill e_i	Decaps: $m =$ codeword closest to ct

Question: are lattices and code the same?

Distance: Euclidean (lattices) vs Hamming (codes).

These metrics are very different but there is some theoretical work on codes using lattice-based theory. One result coming from this is a reduction from finding short codewords to decoding.
(i.e. if I know how to decode I can find short codewords)

Idea: Adapt an equivalent proof reducing from SVP to CVP.

A quick word on formal notions of security

EUF-CMA: Existential UnForgeability under Chosen-Message Attacks
A signature is secure if you can query many signatures from chosen messages and yet fail to create a legitimate signature for another message.

IND-CPA/CCA: INDistinguishability under Chosen Plaintext / Ciphertext Attacks
A PKE/KEM is secure if you can query many encryptions (and decryptions for CCA) and yet fail to distinguish whether a ciphertext comes from one or another message.

EUF-CMA is the gold standard for signatures, and IND-CCA is the gold standard for KEM/PKE.

Real-world cryptography, KEMs

Kyber and HQC's specs are more complicated than these slides!

PKE vs KEM: specs are for KEMs yet they use PKE.

Public key encryption:

- $\mathrm{Keygen}(alea) = (pk, sk)$
- $\mathrm{Encrypt}(pk, m) = c$
- $\mathrm{Decrypt}(sk, c) = m$

Key encapsulation:

- $\mathrm{Keygen}(alea) = (ek, dk)$
- $\mathrm{Encaps}(ek, alea') = (ss, ct)$
- $\mathrm{Decaps}(dk, ct) = ss$

Encapsulation returns both a ciphertext and a session key ss.

Real-world cryptography, KEMs

Kyber and HQC's specs are more complicated than these slides!

PKE vs KEM: specs are for KEMs yet they use PKE.

Public key encryption:

- $\mathrm{Keygen}(alea) = (pk, sk)$
- $\mathrm{Encrypt}(pk, m) = c$
- $\mathrm{Decrypt}(sk, c) = m$

Key encapsulation:

- $\mathrm{Keygen}(alea) = (ek, dk)$
- $\mathrm{Encaps}(ek, alea') = (ss, ct)$
- $\mathrm{Decaps}(dk, ct) = ss$

Encapsulation returns both a ciphertext and a session key ss.

PKE to KEM: Using a PKE to encrypt a random message m yields a KEM ($ss = m$, $ct = c$).

In practice we do it in a more sophisticated way using variants of the so-called **Fujisaki-Okamoto transform.**

Real-world cryptography, FO transform

Fujisaki-Okamoto transform: a generic way to turn an IND-CPA PKE into an IND-CCA KEM, by introducing hash functions.

How it works:

- Derandomization: random values (e.g. r, e_1, e_2) obtained through hash of (random) m and public key.
- Re-encryption: if $m' = \mathrm{Decaps}(dk, ct)$, m' is re-encrypted to check if $\mathrm{Encaps}(ek, m') = (ss, ct)$. If not, Decaps returns a "random" value obtained through hashing (implicit rejection).

Consequence: if an attacker sends an invalid ciphertext, the Decaps function will only return a random string, without explicitly returning a failure.

Caveat: Re-encryption can also open the door for new attacks, especially side-channel attacks (e.g. timing attacks).

Real-world cryptography, decapsulation failure

Decapsulation failure: when ct obtained from Encaps differs from ct after re-encryption, leading Decaps to return the wrong ss.

Is this possible? Why and when?

- Kyber: if sum of errors $> q/4$, $\mathrm{Round}(v - s^T u) \neq \lceil q/2 \rfloor m$
- HQC/BIKE: Decode handles at most Δ errors

Real-world cryptography, decapsulation failure

Decapsulation failure: when ct obtained from Encaps differs from ct after re-encryption, leading Decaps to return the wrong ss.

Is this possible? Why and when?

- Kyber: if sum of errors $> q/4$, $\mathrm{Round}(v - s^T u) \neq \lceil q/2 \rfloor m$
- HQC/BIKE: Decode handles at most Δ errors

Is it a problem?

- **Yes:** on some schemes, there are key-recovery attacks using decapsulation failures.
- **No:** such failures are proven to be extremely unlikely. Also, using ephemeral keys thwarts failure attacks.
- **Trivia:** for BIKE and HQC, extensive work on decoding failure rate (DFR), HQC was selected partly because its DFR was better understood than BIKE's.

An essential operation: polynomial multiplication

At the heart of lattice and code based systems, we find polynomial multiplications in the ring $R = \mathbb{Z}_q[X]/(X^n \pm 1)$.

This operation must be **fast**, but also **secure** as we handle secret data (encapsulated messages or private keys).

First trick: evaluation-interpolation to compute PQ(X)

- Evaluate P and Q on enough points $x_i \in \mathbb{Z}_q$
- Compute all values $P(x_i)Q(x_i)$ (still in $\mathbb{Z}_q$)
- Interpolate to recover PQ (in R)

Advantage: we transform operations in R into operations in $\mathbb{Z}_q$.

Drawback: how to evaluate and interpolate easily?

Idea: $X^n \pm 1$ yields a wise choice for the x_i's.

An analogue of Fourier transform

Fourier: express functions as series in $\omega = e^{\frac{2i\pi}{n}}$, i.e. $\omega^n = 1$.
NTT: evaluation points are ψ^k, where $\psi^n = \mp 1 \bmod q$.
Thanks to that, evaluation is just a matrix product.

Example: consider $q = 7681$, $n = 4$ and
$G(X) = 1 + 2X + 3X^2 + 4X^3$, represented by $(1, 2, 3, 4)^T$
A suitable ψ is either $\psi = 1925$ or $\psi = 5756$.
Evaluating G on powers of ψ amounts to computing

$$\hat{g} = \begin{pmatrix} \psi^0 & \psi^1 & \psi^2 & \psi^3 \\ \psi^0 & \psi^3 & \psi^6 & \psi^9 \\ \psi^0 & \psi^5 & \psi^{10} & \psi^{15} \\ \psi^0 & \psi^7 & \psi^{14} & \psi^{21} \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} \psi^0 & \psi^1 & \psi^2 & \psi^3 \\ \psi^0 & \psi^3 & \psi^6 & \psi^1 \\ \psi^0 & \psi^5 & \psi^2 & \psi^7 \\ \psi^0 & \psi^7 & \psi^6 & \psi^5 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

This comes from evaluating G at $\psi, \psi^3, \psi^5, \psi^7$ then reducing by $\psi^8 = \psi^{2n} = 1$.

Number Theoretic transform (NTT)

More ideas: symmetry $\psi^{k+n} = -\psi^k$ and periodicity $\psi^{k+2n} = \psi^k$

$$\hat{g} = \begin{pmatrix} \psi^0 & \psi^1 & \psi^2 & \psi^3 \\ \psi^0 & \psi^3 & -\psi^2 & \psi^1 \\ \psi^0 & -\psi^1 & \psi^2 & -\psi^3 \\ \psi^0 & -\psi^3 & -\psi^2 & -\psi^1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

Consequence: only small powers of ψ involved + nice pattern (butterflies) $\rightsquigarrow$ efficient implementation.

Interpolation: getting g from $\hat{g}$ in a similar fashion.

Conclusion: we have something fast, but is it secure?

NTT butterflies

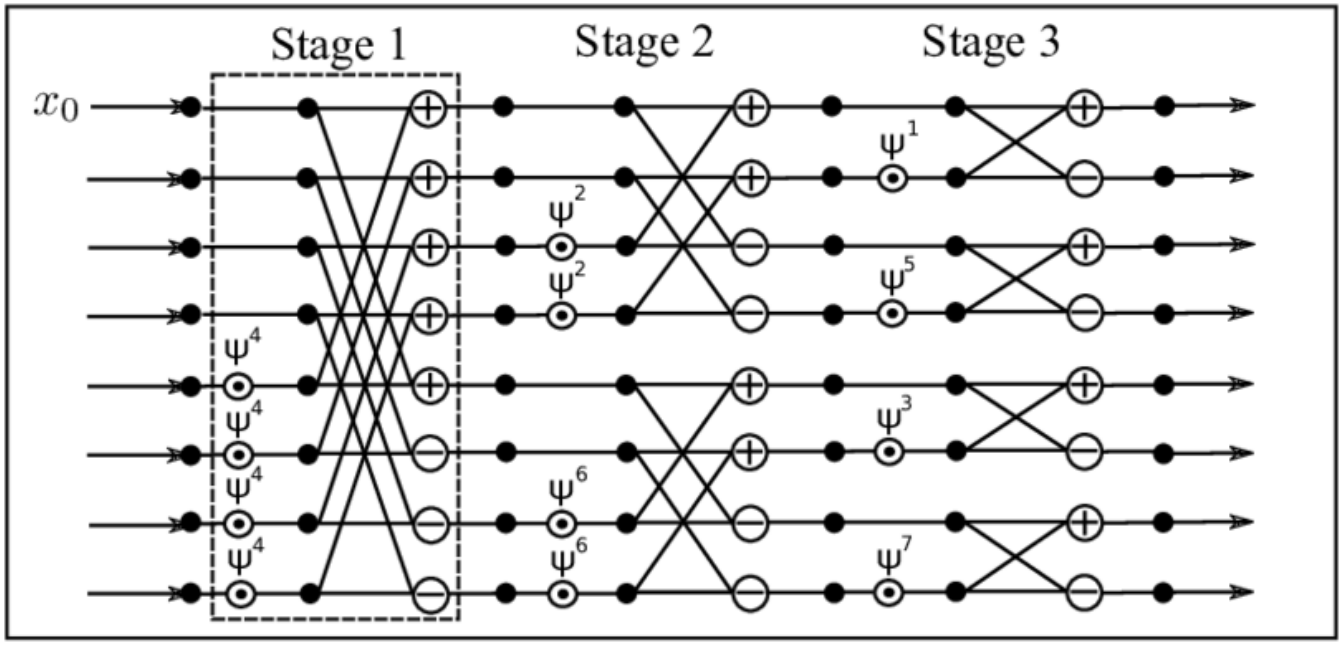


Figure: NTT butterflies for $n = 8$

This may seem complicated but it is super efficient especially using AVX instructions or hardware accelerators.

Side-channel attacks

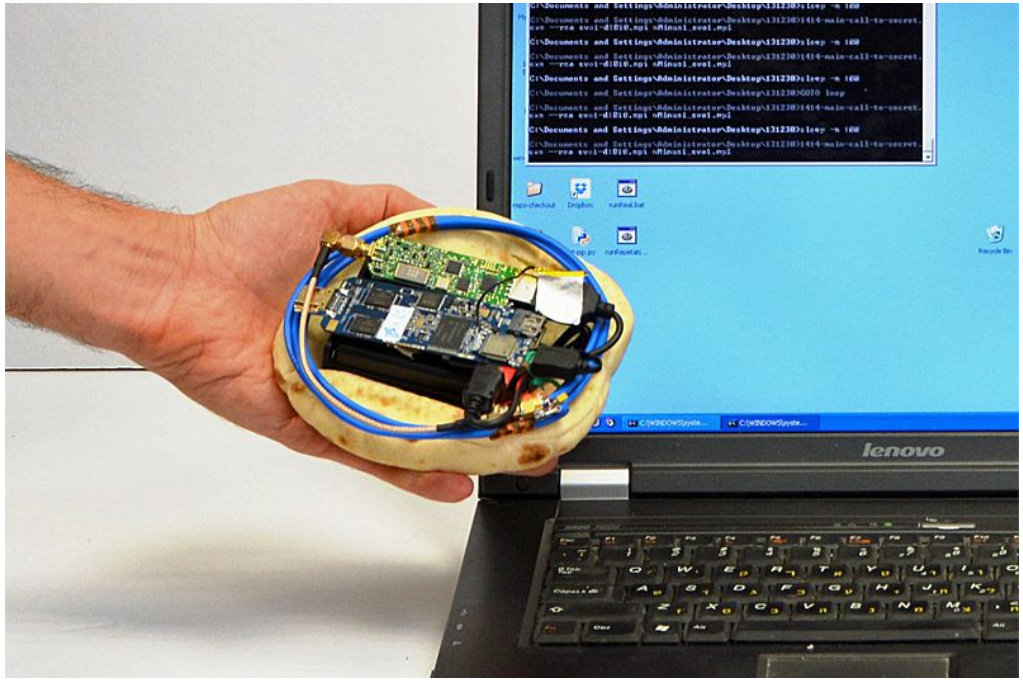


Figure: Portable Instrument for Trace Acquisition (PITA)

Generalities on side-channel attacks

The concept of side channels

Measuring physical quantities (EM radiation, power consumption, heat, etc.) during cryptographic computations can leak secrets.

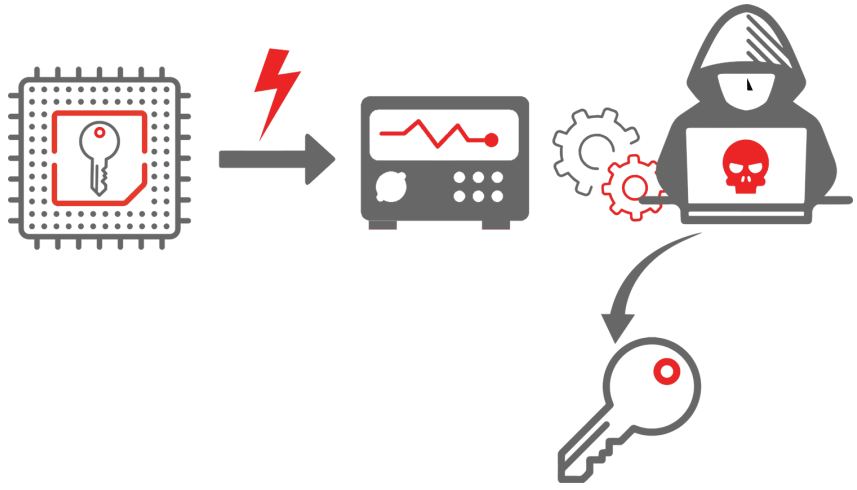


Figure: The principle (Source: Secure IC)

Remark: does not always require physical access to device (e.g. timing attacks, decryption failure oracle).

Topics and trends

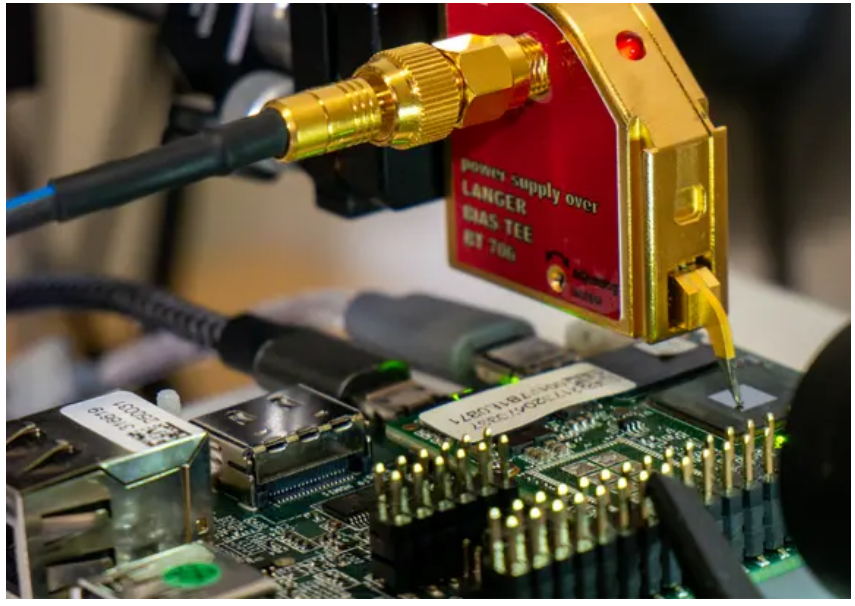


Figure: A probe used for SCA analysis

- Very active topic of research (more than cryptanalysis)
- Many opportunities (previous implementations were not designed for side-channel resistance)
- Efficient tools for acquisition (probes) and exploitation (AI)

A naive example

Consider discrete logarithm cryptosystems. Main operation is scalar multiplication (ECDLP) or exponentiation (FFDLP).

Optimization: use double-and-add strategies to compute $Q = nP$

- if ith bit of n is 1 $Q := 2Q + P$
- if ith bit of n is 0 $Q := 2Q$

Problem: if one can distinguish doubling from adding (e.g. doubling is usually faster), then one recovers the binary decomposition of the secret n.

Remark: also applies to Euclid's algorithm for GCD computation (many conditional branches). Why it matters? Euclid could be used to compute h_0^{-1} in BIKE.

A nice workaround

Montgomery ladder:

- Set $R_0 = 0$, $R_1 = P$
- If ith bit is 0 then $(R_0, R_1) := (2R_0, R_1 + R_0)$
- If ith bit is 1 then $(R_0, R_1) := (R_1 + R_0, 2R_1)$
- Correctness: $R_1 = R_0 + P$ (loop invariant)

Remark: the if clause depends on n but each step = adding and doubling regardless of n.

Why it's nice? A **software** fix and no performance cost.

A taxonomy

Note: SCA are often ad-hoc (and implementation dependent) and comes in a huge variety. They are often classified based on

- their target:
  - signal: sound, power, EM
  - timing attacks
  - cache/memory attacks
  - ... and many more
- the attacker's resources:
  - single-trace attacks (one chance)
  - multi-trace attacks (multiple attempts)
  - template attacks (bought model device)
  - fault injection attacks (tampers with device)

Simple Power Attack (SPA)

Simple power attack (SPA): using one or a few traces, an attacker recovers the secret. Even be doable on the fly without much preparation (depends on noise).

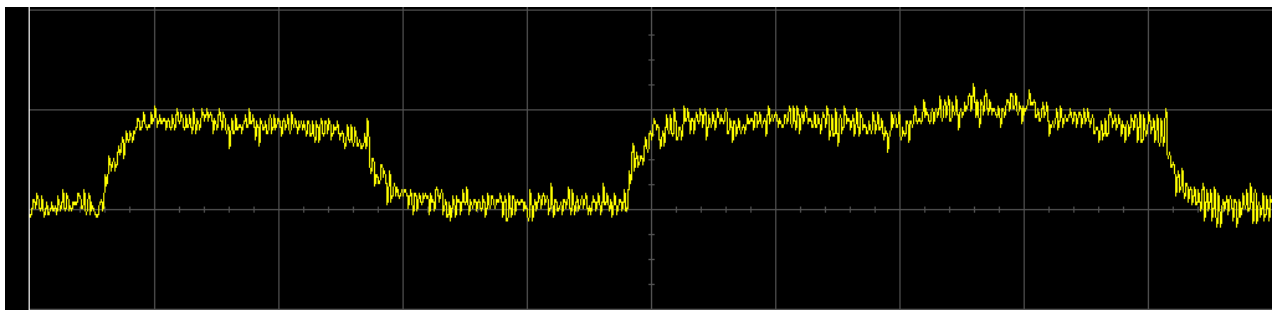


Figure: RSA trace when secret bit is 0 (left) or 1 (right)

Template attacks

Step 1: the attacker has access to the device and first establish models for the power consumption by acquiring many traces.

Step 2: These models are used to predict secret information based on the power trace observed.

Idea: from the observed signal, find the closest template (maximum likelihood). The input yielding this template should be the same as the one who led to the signal.

Remark: This attack is harder to mount but realistic for commercial devices and much more resilient to noise.

An innocent example

Within several lattice-based KEMs (NewHope, Frodo, etc.), an innocuous function encoding bit strings into vectors/matrices.

$$\text{Bitstring } m \mapsto \text{array } m \text{ s.t. } m[i] = \begin{cases} 0 \text{ if } m_i = 0 \\ \lceil q/2^B \rfloor \text{ if } m_i = 1 \end{cases}$$

Innocuous? It parses secret data to be encapsulated!

Step 1: execute Encode(m) for $m = 0\mathrm{x}00, 0\mathrm{x}01, \ldots 0\mathrm{xff}$ and gather templates. Acquire the corresponding 256 traces (in practice maybe 100 traces for each).

Step 2: when the ith byte of m is processed, recover the signal and find the template that fits the observation best.

Result: byte by byte, we recover m or something close enough.

Countermeasure: shuffling

Solution: randomize parsing order.

- Assume m is a 256-bit session key.
- It has $\simeq 128$ bits equal to 1 (m is random)
- There are $\binom{256}{128}$ keys with Hamming weight 128
- This is much more than anyone can bruteforce.

Result: an attacker no longer know which byte of m he observed, so at best he gets the Hamming weight of m.

Warning: here it is fine but sometimes the Hamming weight is sensitive! (e.g. for code-based crypto)

Attacks against NTT

What do I need to compute $\hat{g} = \mathrm{NTT}(g)$?
All $g_i \times \psi^k$, for $g_i \in \mathbb{F}_q$ and $k \leq n/2$.

Idea: there is only $nq/2$ options i.e. $\simeq 10^6$ for Kyber's parameters

Step 1: gather signal traces corresponding to each (g_i, ψ^k)

Step 2: Capture signal during execution and identify the g_i whose template matches observation best (maximum likelihood).

Consequences: within Kyber the product is used

- for encryption in $A^T r$ and $t^T r \rightsquigarrow$ might leak r (and thus m)
- for decryption in $s^T u \rightsquigarrow$ might leak the private key s

Key/message recovery: m is often a session key within a protocol $\rightsquigarrow$ one session compromised.

Attacks against NTT: targets and options

Other targets?

- Dilithium is more challenging, as $nq/2 \simeq 10^{10}$
- Code-based crypto (BIKE, HQC) often avoid NTT
- FrodoKEM does not use the module structure

Countermeasures:

- Blinding: shift secret ($P \mapsto X^\rho \times P$) before product and recover product $X^\rho PQ \mapsto PQ$. Idea: $X^\rho \times P$ leaks and not P. Cheap but not so strong: limited options as $\rho < n$ ($X^n = \mp 1$)
- Shuffling? Would break butterfly structure
- Masking? cf. following slides

Good practices for safe implementations

- Avoid branches and array access depending on secret values
- When no other option, at least ensure that both branches yield to same type and number of operations
- Identify and minimize the code lines where secret is involved
- Randomize when possible: e.g. in $\mathbb{F}_p$ replace x^n by $x^{n+k(p-1)}$ (blinding using $x^{p-1} = 1 \bmod p$.)

More subtle side-channel attacks

Outline

In previous sections :

- Look for operations on secret data (e.g. product)
- Target these operations to recover compromising signals

And if developpers put countermeasures on those operations?

- Look for more subtle places: secret $\neq$ sensitive
- Sensitive data = not secret a priori but actually quite related
- Result often needs to be completed by bruteforce
- If search space is small enough, bruteforce can be an option

The Hamming weight of the syndrome

Code-based: ciphertext $c = mG + e$, syndrome $S = cH^T = eH^T$

Target:

- Is the syndrome sensitive?
- And the weight of S?

The Hamming weight of the syndrome

Code-based: ciphertext $c = mG + e$, syndrome $S = cH^T = eH^T$

Target:

- Is the syndrome sensitive? Yes! Product of $e \rightsquigarrow m$ and H
- And the weight of S?

The Hamming weight of the syndrome

Code-based: ciphertext $c = mG + e$, syndrome $S = cH^T = eH^T$

Target:

- Is the syndrome sensitive? Yes! Product of $e \rightsquigarrow m$ and H
- And the weight of S? The weight of a long vector is not enough (prove it!)

The Hamming weight of the syndrome

Code-based: ciphertext $c = mG + e$, syndrome $S = cH^T = eH^T$

Target:

- Is the syndrome sensitive? Yes! Product of $e \rightsquigarrow m$ and H
- And the weight of S? The weight of a long vector is not enough (prove it!)
- But here we will see that actually it is sensitive

The Hamming weight of the syndrome

Code-based: ciphertext $c = mG + e$, syndrome $S = cH^T = eH^T$

Target:

- Is the syndrome sensitive? Yes! Product of $e \rightsquigarrow m$ and H
- And the weight of S? The weight of a long vector is not enough (prove it!)
- But here we will see that actually it is sensitive

Polynomial point of view: $h(X)$ is secret key

$$S(X) = h(X)e(X) \bmod X^r - 1$$

The Hamming weight of S varies with the value of e
Depending on the "alignment" of e and h it can increase of drop

Exploiting the Hamming weight

- **Active attacker** generates messages m and errors e.
- **Collect traces** leaking the weight of S as e varies
- Deduces where are the 1's in secret key h
- Even with small weight, that's a lot of work

Exploiting the Hamming weight

- **Active attacker** generates messages m and errors e.
- **Collect traces** leaking the weight of S as e varies
- Deduces where are the 1's in secret key h
- Even with small weight, that's a lot of work

- Use the **spectrum** $\mathcal{D}(h)$, i.e. the distances between 1's in h
- Example $h = 1 + X^3 + X^7 + X^8$, modulo $X^{18} + 1$
- **Cyclic distance** is $\min(d, r - d)$
- Spectrum is $\mathcal{D}(h) = \{1, 3, 4, 5, 7, 8\}$
- How? Use $e = 1 + X^d$ for $d \in \{1, \ldots, r/2\}$
- Reconstruct h from spectrum
- Run decapsulation to check that h is correct

The Collision Phenomenon in $\mathbb{F}_2$

- The attacker crafts an error vector containing exactly two active bits separated by a distance d:

$$e(X) = 1 + X^d$$

- The syndrome computed by the target device becomes:

$$S(X) = (1 + X^d) \cdot h(X) = h(X) \oplus X^d \cdot h(X)$$

The Collision Phenomenon in $\mathbb{F}_2$

- The attacker crafts an error vector containing exactly two active bits separated by a distance d:

$$e(X) = 1 + X^d$$

- The syndrome computed by the target device becomes:

$$S(X) = (1 + X^d) \cdot h(X) = h(X) \oplus X^d \cdot h(X)$$

Key Property

S is the XOR of h and a d-**position shifted version** of h

- **If** $d \notin \mathcal{D}(h)$: No overlap $\implies$ Maximal Hamming weight
- **If** $d \in \mathcal{D}(h)$: Overlap $\implies$ $1 \oplus 1 = 0$ $\implies$ **Weight drop!**

Case 1: The distance d is NOT in the secret key

Side-Channel Observation

No active bit of $h(X)$ aligns with an active bit of the shifted key

The Hamming weight of S is maximal: $\mathrm{wt}(S) = 2\,\mathrm{wt}(h)$

```
h(X)           :  1  0  1  0  0  0  0  ...  (Weight = 2)
X^d * h(X)     :  0  0  0  0  1  0  1  ...  (Shifted by d)
-----------------------------------------
Syndrome (S)   :  1  0  1  0  1  0  1  ...  (Weight = 4)
```

$\Longrightarrow$ **Power consumption signal is maximal**

Case 2: The distance d is in the secret key

Side-Channel Observation

A pair of active bits in $h(X)$ is separated by exactly d

Some bits are cancel by the shifted bits ($1 \oplus 1 = 0$)

```
h(X)           :  1  0  0  1  0  0  0  ...  (Weight=2)
X^d * h(X)     :  0  0  0  1  0  0  1  ...  (Shifted by d=3)
-------------------------------------------
Syndrome (S)   :  1  0  0  0  0  0  1  ...  (Weight = 2)
                             ^
                    Collision (1 + 1 = 0)
```

Case 2: The distance d is in the secret key

Side-Channel Observation

A pair of active bits in $h(X)$ is separated by exactly d

Some bits are cancel by the shifted bits ($1 \oplus 1 = 0$)

```
h(X)           :  1  0  0  1  0  0  0  ...  (Weight=2)
X^d * h(X)     :  0  0  0  1  0  0  1  ...  (Shifted by d=3)
-------------------------------------------
Syndrome (S)   :  1  0  0  0  0  0  1  ...  (Weight = 2)
                             ^
                      Collision (1 + 1 = 0)
```

Physical Consequence

The syndrome weight drops, it translates into a measurable **power consumption dip** or a **timing variation**

Final reconstruction

- **From the spectrum $\mathcal{D} = (d_1, \ldots, d_{w-1})$ to h**
  - Set $h_0 = 1$ and $h_{d_1} = 1$ (doable by cyclicity of h)
  - Find all $k > d_1$ such that $h_k = 1$ compatible with $\mathcal{D}$
  - Each possible k is a node in a tree rooted at d_1
  - Repeat the process at each node to build the next level
  - When a contradiction occurs, prune the tree
  - **Exercise:** compute the complexity (remember w is small)
- **Limits:**
  - When d appears twice (or more) in $\mathcal{D}$, can we detect it?
  - More generally, the traces must be accurate (not missing a d)
  - Obviously, only works for **static keys** (why?)

Other options to use the Hamming weight

- **Fault-injection:**
  - Flip $h_i \leftarrow h_i \oplus 1$ using a laser or EM pulse
  - Observe $wt(S)$ and deduce the initial value of h_i
  - Of course, requires precision to flip h_i and not h_{i+1}
- **Correlation attacks:**
  - Observe many computations $S = e \cdot h$
  - On similar device, run $S = e \cdot h$ for many known $h^{(i)}$
  - Correlate observations with **templates** $h^{(i)}$ to get h
  - Only doable on small chunks of h (e.g. byte per byte)

Another example on lattices

- Sometimes, even more subtle changes are used
- In Kyber decapsulation, round $v - \langle u, s \rangle$ ($v - s^T u$ here)
- **Recall: we work in $\mathbb{F}_q$**

Arithmetic modulo p :

- $v \in \mathbb{F}_p$ represented modulo p, i.e.
- $v \in \{0, \dots p-1\}$ ou $v \in \left\{-\frac{p-1}{2}, \dots, \frac{p-1}{2}\right\}$
- There is an $A \in \mathbb{F}_p$ s.t. $A + 1 = 0 \rightsquigarrow$ change of behavior

Actual parameters for Kyber

$v \in \mathbb{F}_q[X]/X^n + 1$ is a vector $v = (v_1, \dots, v_n)$ with $v_i \in \mathbb{F}_q$

How to mount an attack

Active attacker: manipulates v and u as he wishes

- Start with $u = (1, 0, \ldots, 0)$
- Target computes $v - s^T u = v - s_1$
- Repeat operation for increasing values v
- Change of behavior when $v = s_1$ (or $s_1 + \frac{p-1}{2}$)
- Repeats with $u = (0, 1, \ldots, 0)$ to get s_2

Cost:

- Getting $s_1 \rightsquigarrow \leq 2\eta + 1$ attempts ($s_i \in [-\eta, \eta]$)
- Retrieving $s \rightsquigarrow 2nk\eta$ attempts
- For Kyber $k = 2$, $n = 256$, $\eta = 3$ & $q = 3329$, i.e. $nk(2\eta + 1) = 3584$
- At fairly high speed (100 decaps/s) it would take 36s
- At highest security (level V), $k = 4$ but $\eta = 2$ so the cost is < 1 min

Ideal countermeasure: use ephemeral keys whenever possible!

Masking and shuffling



Beyond good practices

Even good practices are not sufficient to prevent intrinsic leakage. In practice, several options are considered:

- **Emission control:** use Faraday cages, measure EM emissions and confirm that they are unusable (e.g. for satellites) or make sure that no attacker can be physically too close to cryptographic devices (NATO TEMPEST)
- **Parasite emissions:** generate noise, or run tasks in parallel to make it harder to identify the relevant signal
- **Shuffling & masking:** ensure that leakage is not directly related to secret information

First option has an impact on device cost, last two have an impact on device performance (not unusual to have factors 10 to 100).

Shuffling

Idea: when performing operations on the coordinates of a secret vector x, for instance to compute $\sum_i a_i x_i$, randomize the order in which the $a_i x_i$'s are computed and summed.

Why? If $x_i = 0$, $a_i x_i$ is much easier to compute, so an attacker could identify zeroes in x (thus x if $x \in \mathbb{F}_2^n$). If the order is randomized, no way of knowing which x_i is 0.

Limits:

- Heavy cost (generating random permutations, deactivating compiler/hardware optimization)
- Not always sufficient (in the above example, the Hamming weight of x still leaks)

Masking

- **Basic masking:** pick random mask m and perform operations on $x \oplus m$ to ensure that leakage is not related to x anymore. Can also be done with arithmetic operations (Boolean vs arithmetic masking).
- **ISW masking:** cut the secret x into slices and XOR the slices with masks. Invented by Ishai, Sahai and Wagner in 2003, current main standard for masking.

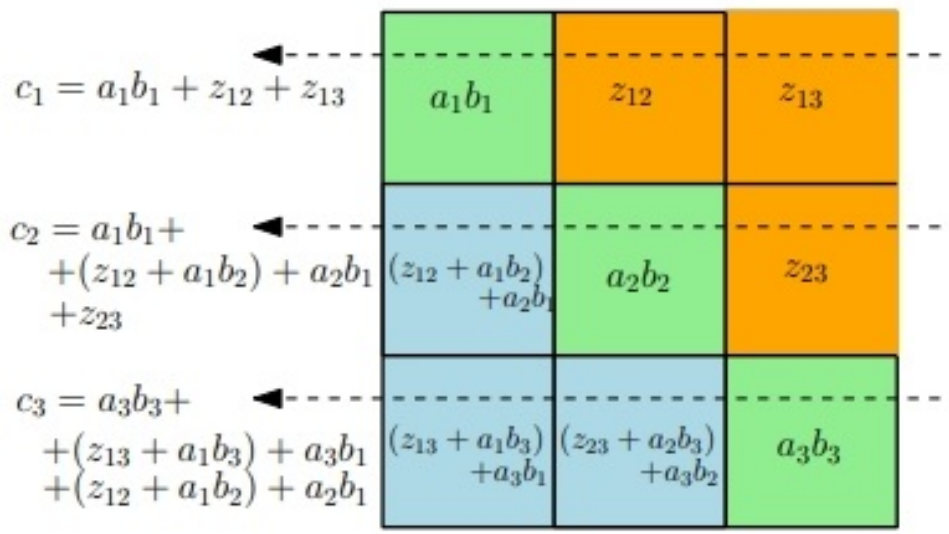
Limits:

- Heavy cost (even worse than shuffling)
- Composition of masked primitives is not always secure
- Security model assume perfect hardware and randomness

Masking, a small example

Objective: Securely compute $c = ab$, where $a = a_1 \oplus a_2 \oplus a_3$ and $b = b_1 \oplus b_2 \oplus b_3$ are both secret, by computing slices c_1, c_2, c_3 such that $c = c_1 \oplus c_2 \oplus c_3$.

Option 1: using random z_{ij}
Option 2: using threshold
$c_1 = a_2 b_2 \oplus a_2 b_3 \oplus a_3 b_2$
$c_2 = a_1 b_3 \oplus a_3 b_1 \oplus a_3 b_3$
$c_3 = a_1 b_1 \oplus a_1 b_2 \oplus a_2 b_1$
(c_i independent of a_i, b_i).



In both cases, one can prove that observing one share c_i is not enough to completely recover a or b.

Higher order masking

Depending on the model, a d-order masked implementation refers to:

- A masked implementation that is secure against order d power analysis
- A threshold scheme in which an attacker needs at least $d + 1$ shares to recover secret values

High-order masking is not an easy task as it incurs a significant computational cost. Some designs were proposed to reduce the cost (e.g. use less randomness) but many failed.

Though masking aims at making side-channel attacks theoretically impossible, in practice even first order masking makes them much harder to perform.

Masking efforts in practice

The various flavors of masking:

- Cheap techniques providing some security at virtually no cost: scalar / base point blinding, projective coordinates with $Z \neq 1$
- So-called "gadgets" for limited sensitive operations (e.g. product, sum) provide security at a price.
- End-to-end masked implementation: assembling gadgets to cover all sensitive parts of an algorithm. Secure but extremely expensive (not sure it is used in practice)

Current trends in masking:

- Provide gadgets for asymmetric crypto, especially post-quantum (lattices + BIKE so far)
- Schemes that designed for easier masking (e.g. Raccoon, an analogue of ML-DSA)

Beyond SCA, fault-injection attacks

Taxonomy: voltage glitches, clock glitches or using devices (lasers, EM fields) to force operations on tampered data.

Examples: zero-value attacks, nonce reuse, RowHammer attack on ARM/Android in 2016 (hammering in DRAM cells).

Limits: access to device, static keys, expensive equipment.

Scenarios: widely spread devices on which we can experiment, or powerful attacker (state), e.g. on recovered military equipment.

Countermeasures: expensive hardware countermeasures (e.g. shielding device to prevent tampering, or ensuring that any fault injection attempt destroys the device).

Introduction to cryptanalysis



Attacks without side-channels

Cryptanalysis: breaking schemes using no side-channel (i.e. respecting the attacker models EUF-CMA, IND-CCA, etc.)

What it takes: sophisticated algorithms (number theory), smart observations, experience, knowledge of past mistakes

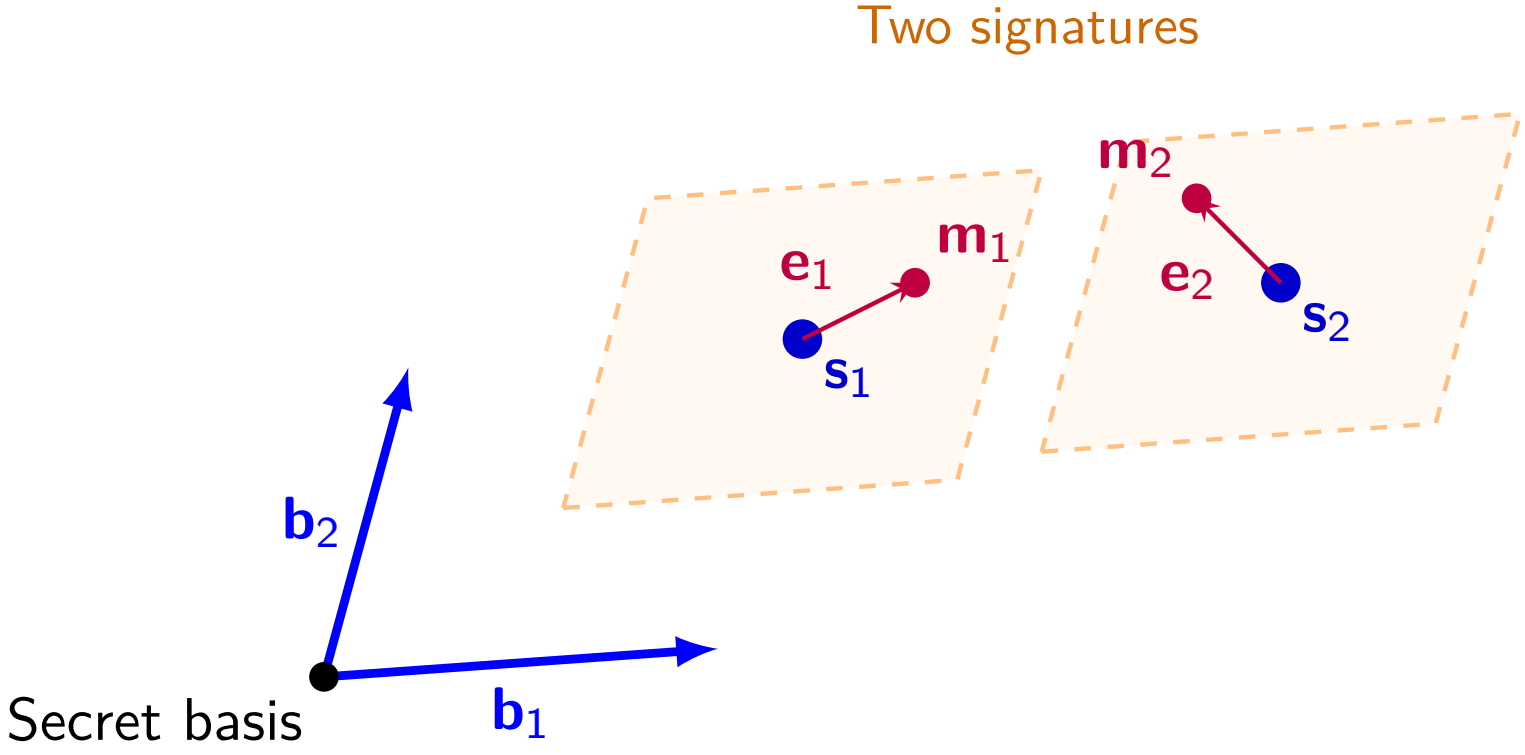
Small example: the hidden parallelepiped

- Target lattice-based signature scheme
- From (public) signatures retrieve secret key
- No other information required

Lattice signatures

Summary:

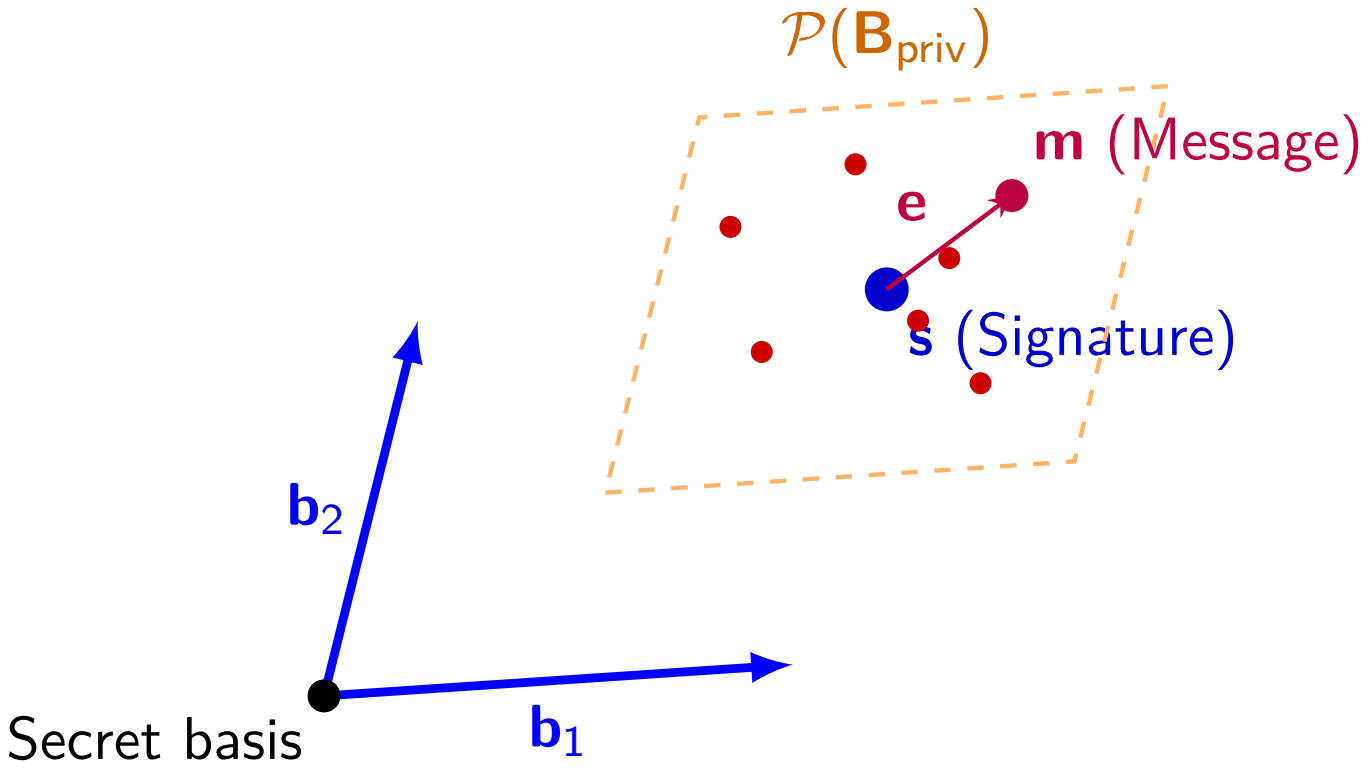
- Secret key: good basis allowing to compute closest point
- Without secret key, this task is hard (solving CVP)
- Message $m_i \rightsquigarrow$ signature s_i is the closest point in the lattice
- **Idea:** $e_i = s_i - m_i$ is in $B \cdot \left[-\frac{1}{2}, \frac{1}{2}\right]^n$ (parallelepiped)



Shifting everything to one cell

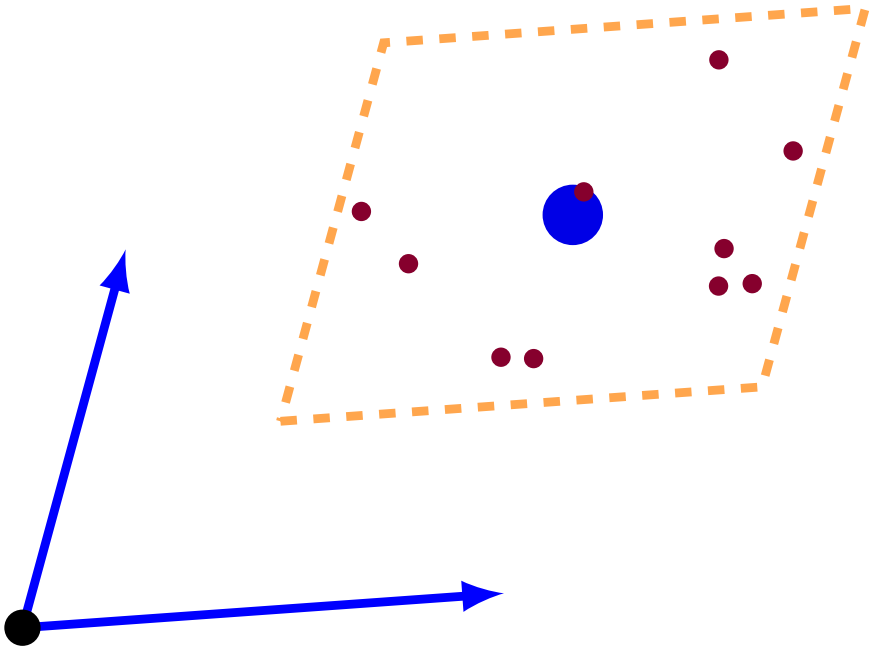
Idea: gather all info in one cell

- Each signature is a point of the lattice
- Translation $\rightsquigarrow$ map all signatures to one point (here $(1, 1)$)
- All messages m_i now live in the same cell (parallelepiped)



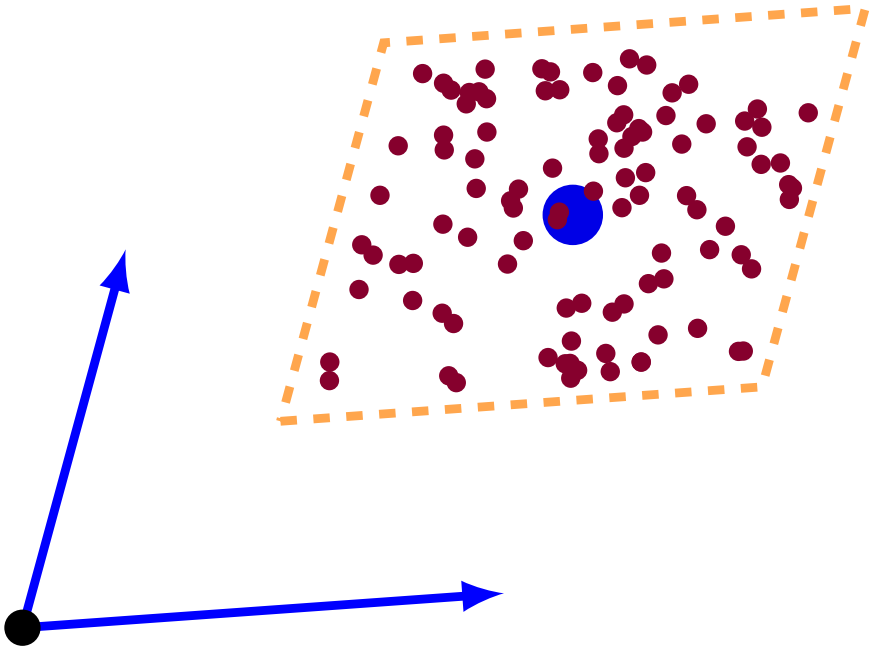
Gathering more and more information

With 10 signatures, not much risk



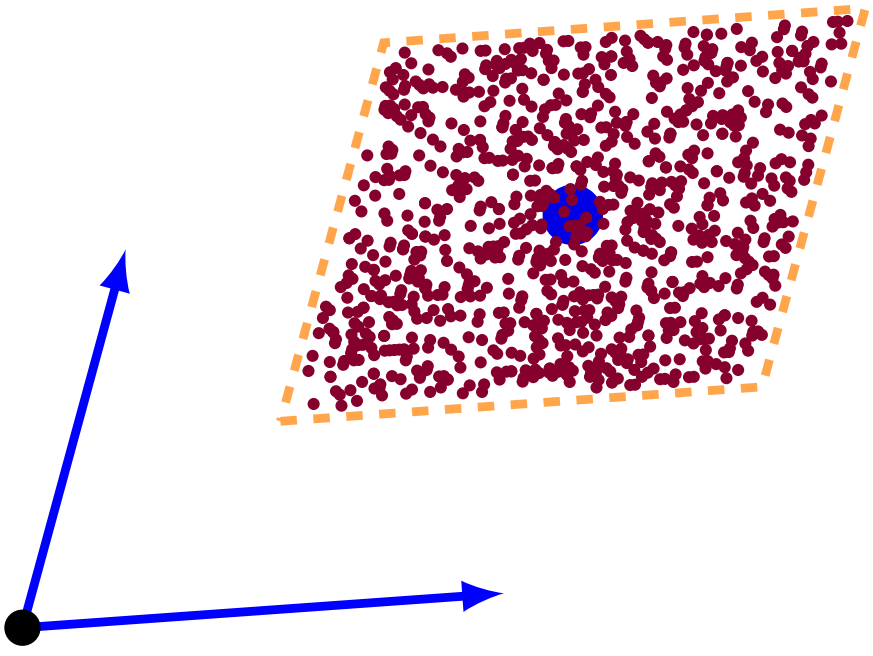
Gathering more and more information

With 100 signatures, a rough idea of the cell



Gathering more and more information

With 1000 signatures, we recover the secret basis



More on the hidden parallelepiped

Does it really work with real crypto?

- Attack due to Nguyen-Regev (2006) against GGH and NTRU
- EUF-CMA model: attacker only gets signatures and messages
- Needs $O(n \log n)$ signatures with n dimension of lattice
- In crypto, $n \approx 500$ to 1000, in practice < 100000 sigs needed

How to avoid it?

- It relies on **deterministic** rounding, i.e. $e_i \in \left[-\frac{1}{2}, \frac{1}{2}\right]^n$
- FN-DSA (Falcon) uses Gaussian sampling
  (parallelepiped becomes a ball giving much less info)
- ML-DSA (Dilithium) uses rejection sampling
  (breaks statistical link between key and signature)

Beyond cryptography: the world of cybersecurity

Cybersecurity is a complex ecosystem where many constraints and actors have a say. **Cryptography is one aspect among others!**

- **Key management:** distributing, updating and storing keys is a major challenge involving complex solutions.
- **Secure software:** even the strongest algorithm is useless if poorly implemented or if there is an interface bug. Such flaws are not easy to detect!
- **Hardware security:** physical attacks (e.g. arson) against servers, fault injection and side-channel attacks, memory attacks, you name it...
- **Culture of security:** is the user trained, aware, careful? Cryptography can do nothing if the password is $\mathrm{Hell0world}$.